

Rethinking Agentic Search with PI-SERINI: Is Lexical Retrieval Sufficient?

Tz-Huan Hsu
University of Waterloo

Jheng-Hong Yang
Stencilzeit

Jimmy Lin
University of Waterloo

Abstract

Does a lexical retriever suffice as large language models (LLMs) become more capable in an agentic loop? This question naturally arises when building deep research systems. We revisit it by pairing BM25 with frontier LLMs that have better reasoning and tool-use abilities. To support researchers asking the same question, we introduce PI-SERINI, a search agent equipped with three tools for retrieving, browsing, and reading documents. Our results show that, on BrowseComp-Plus, a well-configured lexical retriever with sufficient retrieval depth can support effective deep research when paired with more capable LLMs. Specifically, PI-SERINI with gpt-5.5 achieves 83.1% answer accuracy and 94.7% surfaced evidence recall, outperforming released search agents that use dense retrievers. Controlled ablations further show that BM25 tuning improves answer accuracy by 18.0% and surfaced evidence recall by 11.1% over the default BM25 setting, while increasing retrieval depth further improves surfaced evidence recall by 25.3% over the shallow-retrieval setting. Source code is available at <https://github.com/justram/pi-serini>.

1 Introduction

Retriever effectiveness is widely treated as the hard ceiling of information-seeking systems such as Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) and deep research (White, 2024; Team et al., 2025; Li et al., 2025). This view has motivated retrievers that can reason over complex queries (Zhang et al., 2025; Shao et al., 2025; Chen et al., 2026), rather than relying on vanilla lexical retrievers such as BM25. However, as frontier LLMs become increasingly capable of reasoning and tool use, these systems increasingly operate through an agentic loop, where LLMs receive feedback from their environments (Yao et al., 2023). Under this interaction pattern, retriever effective-

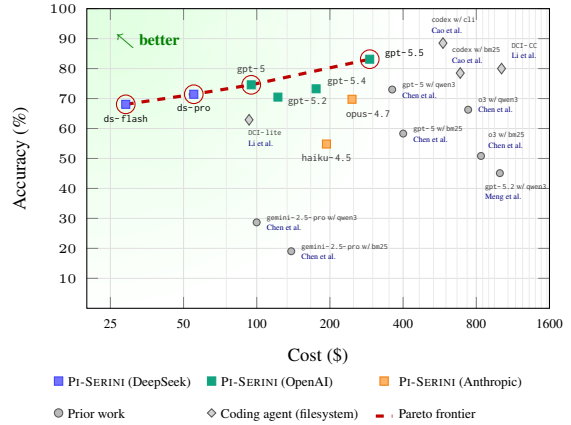


Figure 1: Accuracy vs. Cost trade-off on BrowseComp-Plus. ds abbreviates deepseek-v4 model, and qwen3 abbreviates qwen3-embed-8b retriever. All PI-SERINI systems are paired with BM25.¹

ness may no longer be the only component worth optimizing at all costs. We therefore revisit a central question in building such systems: *Does a lexical retriever suffice as LLMs become more capable in an agentic loop?* Without answering it, we again risk overemphasizing retriever improvement while overlooking other opportunities (Yang et al., 2019).

We begin by testing whether previous lexical baselines were merely ill-configured or whether they are genuinely incapable of finding relevant documents. We verify the BM25 parameter settings and the retrieval depth to increase the likelihood that relevant documents remain in retrieved results, rather than being discarded before they are returned to and processed by the LLM.

However, because LLMs are often constrained by limited context windows, they need an interface that helps them extract useful information for subsequent actions. We therefore introduce PI-SERINI, a deliberately minimal search agent. The agent is equipped with three search-related tools—search, read_search_results, and read_document—so

¹Results from Cao et al. (2026) are based on sampled evaluations and provided for reference. See Section 7.

that document retrieval, result browsing, and document reading become distinct decisions. By leveraging the tool-use abilities of LLMs, PI-SERINI uses this tool interface to cache a retrieved ranking, explore deeper retrieval results, and decide which evidence enters its context window.

In addition, deep research is inherently time-sensitive, requiring systems to balance answer quality against realistic latency and cost constraints. To better match user-facing deep research workloads, we use time-budget steering instead of the fixed iteration cap used in prior work (Chen et al., 2025; Meng et al., 2026), allowing the agent to terminate under a hard wall-clock time budget.

On BrowseComp-Plus (Chen et al., 2025), under time-budget steering, PI-SERINI with gpt-5.5 achieves 83.1% answer accuracy and 94.7% surfaced evidence recall, outperforming released search agents based on dense retrievers. Ablations show that BM25 tuning improves answer accuracy by 18.0% and surfaced evidence recall by 11.1% over the default BM25 setting, while increasing retrieval depth further improves surfaced evidence recall by 25.3% over the shallow-retrieval setting.

Finally, PI-SERINI benefits from a prefix-cache-friendly agentic loop, allowing many repeated input tokens to be served from cache and making prefix caching central to its cost efficiency. BM25 tuning and wall-clock time budgeting provide additional cost savings. Figure 1 summarizes the resulting accuracy–cost trade-off, showing that PI-SERINI consistently lies in the efficient region.

Overall, the BrowseComp-Plus experiments suggest that, with more capable LLMs in an agentic loop, BM25 can be sufficient for effective deep research when it is well-configured, used at sufficient retrieval depth, and paired with a tool interface that helps the agent manage cached retrieval ranking. Our contributions are as follows:

- We introduce PI-SERINI, a search agent that manages cached rankings for deeper retrieval and selective evidence acquisition.
- We reassess BM25 on BrowseComp-Plus by separating lexical retrieval capacity from configuration and retrieval depth, showing that both greatly affect answer accuracy and evidence recall.
- We show that, on BrowseComp-Plus, BM25-based search agents can match or outperform dense-retriever agents, while reducing evaluation cost by $3.3\times-10\times$.

2 Related Work

Deep Research. Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) grounds answer generation in external documents, but complex information-seeking tasks often require more than a single retrieval step. They involve iterative search, document inspection, and evidence synthesis across multiple sources, motivating search agents that extend RAG into multi-step retrieval-and-reasoning processes (Shi et al., 2025; Team et al., 2025; Asai et al., 2024). Recent work has improved these agents through reasoning enhancement (Zhang et al., 2026; Wen et al., 2026; Shao et al., 2024) and tool-use optimization (Wu et al., 2024; Qian et al., 2026). As these systems become more capable, their evaluation has also shifted toward more realistic research settings: Benchmarks such as BrowseComp (Wei et al., 2025) and BrowseComp-ZH (Zhou et al., 2025) evaluate open-web browsing ability, while BrowseComp-Plus provides a fixed-corpus setting for controlled comparison of search agents (Chen et al., 2025). These benchmarks make retrieval and evidence interaction central to agent performance, raising the question of how much observed performance depends on the retrieval model itself.

Retriever for Deep Research. Search agents often use dense retrievers, sparse retrievers, and rerankers to improve early-stage ranking accuracy (Zhang et al., 2025). Recent reasoning-aware retrievers further target complex queries by capturing implicit intent and resolving multi-hop information needs (Shao et al., 2025; Chen et al., 2026). By contrast, lexical methods such as BM25 are commonly used as non-dense baselines, and their weaker end-to-end performance is often attributed to semantic mismatch.

Related analyses have examined retrieval components in search agents: SAGE (Hu et al., 2026) studies retrieval effectiveness with LLM-based retrievers, while other work (Sharifmoghaddam and Lin, 2026; Meng et al., 2026) studies how reranking and document ordering affect search agents in deep research. However, these studies do not isolate whether BM25’s apparent weakness comes from lexical retrieval itself or surrounding design choices such as configuration and retrieval depth. Our work addresses this gap by separating BM25 capacity from these surrounding choices. We therefore revisit the question: *Does a lexical retriever suffice as LLMs become more capable in an agentic loop?*

3 PI-SERINI

PI-SERINI is a search agent for isolating agent–retriever interaction. The LLM agent runs a ReAct loop, while a retrieval controller mediates all access to an ANSERINI BM25 backend. This controller is the main isolation point: it exposes a constrained tool API, maintains cached ranking from retrieval, and controls how retrieved evidence enters the agent context. Appendix A gives the exact prompt, backend configuration, cache behavior, and run-time policy used in our experiments.

3.1 Preliminary

We formulate deep research as an iterative retrieval-and-reasoning problem in a ReAct-style agentic loop (Yao et al., 2023). Given a user query, a search agent, commonly materialized by an LLM, iteratively reasons, calls tools, observes their outputs, and eventually returns a final response.

The agent operates over an interaction trajectory:

$$\mathcal{H}_T = (\tau_1, a_1, o_1, \dots, \tau_t, a_t, o_t, \dots, \tau_T, a_T)$$

where τ_t is the reasoning trace generated at turn t , a_t is the action selected by the agent, and o_t is the observation returned by the environment. At each turn, the agent policy π samples a reasoning trace and an action conditioned on the previous interaction history:

$$\tau_t, a_t \sim \pi(\cdot \mid \mathcal{H}_{t-1})$$

For intermediate turns ($t < T$), the action a_t is a tool call or reasoning, and the environment returns an observation o_t . The final action a_T produces the response and terminates the loop. We treat the outputs as both the final response R and the set of documents D observed during the retrieval process.

3.2 Components

PI-SERINI has three design elements. First, it separates retrieval, result browsing, and document reading into distinct tool calls. Second, it records the agent’s interaction trajectory at multiple evidence-access levels. Third, it uses time-budget steering to ensure that the agent answers before timeout.

Retrieval Controller. Naive search agents often use shallow retrieval settings that insert the full texts of all ranked documents directly into the context window. This removes the need for explicit evidence selection, but conflates retrieval depth with context management and makes deeper retrieval

impractical. In contrast, PI-SERINI leverages LLM tool use through separate tools for retrieval, browsing, and reading, allowing the agent to cache ranked documents and selectively control which evidence enters its context window. Our tool interface follows the workflow in Appendix A.2: search first, browse the cached ranking when useful, and selectively read promising documents. The action a can be one of three search-related tool calls:

- **search:** issues a lexical query, retrieves up to 1000 ranked documents, caches the ranking under a session-local `search_id`, and exposes only the top 5 excerpts on the initial call;
- **read_search_results:** browses a cached ranking by `search_id` using rank pagination, allowing the agent to inspect results without issuing a new backend query;
- **read_document:** fetches one document by identifier using line-based pagination, allowing the agent to read only the relevant text pieces.

Trajectory Logging. Each run produces a structured per-query artifact that records the full tool call and reasoning trajectory. Rather than tracking only a single document set returned by the search tool, PI-SERINI records four document sets:

- D_{surfaced} : documents returned by search.
- $D_{\text{previewed}}$: documents whose excerpts were shown through `read_search_results`.
- D_{opened} : documents read via `read_document`.
- D_{cited} : documents cited in the final answer.

These tiers distinguish what the system made available, what the agent chose to inspect, and what it ultimately used.

Time-Budget Steering. We impose a timeout rather than a fixed iteration cap on the agentic loop. Each query run is subject to a timeout of T seconds.

To encourage completion before hard termination, PI-SERINI uses a two-stage time-budget policy. At $0.7T$, the system injects a submission steer that instructs the agent to stop using tools and produce its best answer from the evidence collected so far. From that point onward, all three tools are blocked. If the agent still does not submit by time T , the process is terminated and the query is marked as timed out.

4 Experimental Setup

We study our research question—*Does a lexical retriever suffice as LLMs become more capable in*

an agentic loop?—by evaluating search agents that pair different LLMs with either dense retrievers or BM25, measuring both retrieval effectiveness and final answer quality.

4.1 Evaluation Settings

Dataset. We evaluate PI-SERINI on the widely used BrowseComp-Plus (Chen et al., 2025), a deep research benchmark consisting of 830 queries and 100,195 documents. On average, each query is associated with 6.1 evidence documents and 2.9 gold documents. Each document contains an average of 5,179.2 words and 32,296.2 characters.

Evidence documents are documents required to answer the query, while gold documents are a stricter subset that both support answering and semantically contain the final answer.

Baselines. We compare PI-SERINI with several representative BrowseComp-Plus baseline systems. The released systems pair o3 and gpt-5 with either qwen3-embed-8b (Zhang et al., 2025) or BM25. We also report the baseline from the work of Meng et al. (2026), which also uses qwen3-embed-8b retriever but replaces the reasoning model with gpt-5. 2. Finally, we report the numbers of AgentIR (Chen et al., 2026), a reasoning-intensive retriever trained for deep research.

Unlike PI-SERINI, which exposes separate search, read_search_results, and read_document tools, the baseline agents use a single retriever tool that directly returns the top- k search results, with $k = 5$. Under our logging protocol, we therefore only consider D_{surfaced} for baseline agents, since they do not use additional tools beyond search unless specified otherwise.

Metrics. Following BrowseComp-Plus’s evaluation settings, we report both answer-quality metrics and retrieval-behavior metrics. For answer evaluation, we use an LLM judge. For each query, the judge receives the question, the agent’s final response, and the benchmark-provided correct answer, then determines whether the extracted final answer is semantically equivalent to the correct answer. The judge is run with gpt-5.3-codex; Appendix A.5 gives the exact configuration and prompt. Specifically, we report:

- Accuracy: the fraction of queries whose final answers are judged correct by the LLM judge;
- Calibration Error: the discrepancy between the model’s confidence and empirical correctness.

For retrieval evaluation, we report recall over the logged document sets introduced in Section 3.2:

- Surfaced Recall: recall computed over the document set D_{surfaced} ;
- Previewed Recall: recall computed over the document set $D_{\text{previewed}}$;
- Behavior Recall: recall computed over the union of the document sets $D_{\text{opened}} \cup D_{\text{cited}}$.

4.2 PI-SERINI Implementation Details

Retriever. Documents are indexed using BM25 via ANSERINI (Yang et al., 2018) over the BrowseComp-Plus corpus. We use tuned BM25 parameters with $k_1 = 25$ and $b = 1$, which are selected to better support long-document retrieval in a deep research setting. The BM25 retrieval depth is set to 1000. This retrieval configuration is held fixed across all PI-SERINI runs. Further details on BM25 tuning are provided in Section 6.

Models. We evaluate PI-SERINI using a diverse set of frontier LLMs with better reasoning and tool-use abilities, reflecting the class of sufficiently capable LLMs considered in our research question. For DeepSeek, we use deepseek-v4-flash and deepseek-v4-pro. For Anthropic, we use claude-haiku-4.5 and claude-opus-4.7. For OpenAI, we use gpt-5, gpt-5.2, gpt-5.4, gpt-5.4-mini, and gpt-5.5. This model set across different providers allows us to evaluate whether PI-SERINI’s design remains effective across different LLM families and scales.

Agent Harness. We build PI-SERINI on top of PI (Zechner and contributors, 2025), a minimal but malleable agent harness without sub-agents or additional orchestration modules. We repurpose PI as our search agent by removing its system prompts and built-in tools, then design our own. We configure it with our deep research prompt that instructs it to issue queries, inspect retrieval results through our tools, and synthesize an answer. Per-query timeout is set to $T = 300$ seconds.

5 Main Experimental Results

Answer Quality. Table 1 reports the answer accuracy of all systems. We first compare systems that use the same LLM, gpt-5, with BM25. Under this setting, PI-SERINI improves answer accuracy from 58.3% for the released gpt-5 + BM25 baseline in row (c) to 74.6% in row (i). It also slightly exceeds the released gpt-5 + qwen3-embed-8b

LLM	Retriever	Answer Quality			Surfaced Recall		Previewed Recall		Behavior Recall		
		Acc.	Calib.	Cost (\$)	Evi.	Gold	Evi.	Gold	Evi.	Gold	
Chen et al. (2025) [†]											
(a)	o3	bm25	50.8	39.1	836.4	56.6	61.7	-	-	-	-
(b)	o3	qwen3-embed-8b	66.3	32.7	740.8	73.2	76.3	-	-	-	-
(c)	gpt-5	bm25	58.3	13.5	400.4	61.7	66.5	-	-	-	-
(d)	gpt-5	qwen3-embed-8b	73.0	9.7	360.7	79.0	81.3	-	-	-	-
Meng et al. (2026) [‡]											
(e)	gpt-5.2	qwen3-embed-8b	45.1	-	1 ~ 2k	-	74.7	-	-	-	-
Chen et al. (2026) [‡]											
(f)	Tongyi-DR	AgentIR-4B	68.1	-	-	79.2	-	-	-	-	-
PI-SERINI (ours)											
(g)	claude-haiku-4.5	bm25	54.8	17.3	193.5	94.1	95.4	58.2	60.1	40.9	46.3
(h)	claude-opus-4.7		69.8	10.2	246.6	81.2	86.8	43.3	52.5	30.4	43.0
(i)	gpt-5		74.6	7.2	94.9	90.5	93.8	62.7	70.0	45.7	56.2
(j)	gpt-5.2		70.5	6.2	122.2	89.9	92.8	60.5	67.3	44.8	54.5
(k)	gpt-5.4-mini		68.1	13.7	86.2	91.9	94.1	60.1	65.4	43.1	52.1
(l)	gpt-5.4		73.3	9.2	175.5	93.8	95.3	70.3	66.9	51.8	58.1
(m)	gpt-5.5		83.1	15.7	291.6	94.7	94.4	73.6	72.9	58.9	56.1
(n)	deepseek-v4-flash		68.1	15.5	28.9	94.5	95.7	67.9	69.9	55.2	60.6
(o)	deepseek-v4-pro		71.4	7.0	55.1	91.3	92.5	60.0	63.0	45.4	50.6

Table 1: Answer quality, retrieval behavior, and cost on BrowseComp-Plus.

[†] Results from [Chen et al. \(2025\)](#) are obtained by evaluating their released runs using our own judge.

[‡] Results from [Meng et al. \(2026\)](#) and [Chen et al. \(2026\)](#) are taken from the reported numbers in the original papers.

dense-retriever baseline in row (d). This comparison suggests that previously released baselines understate the potential of BM25, and that improving its configuration and increasing retrieval depth makes it a competitive alternative to dense retrievers for deep research.

Among BM25-based systems, PI-SERINI with gpt-5.5 in row (m) achieves the highest accuracy, at 83.13%. This is also the highest accuracy in Table 1, further showing that a BM25-based search agent can achieve highly competitive performance when BM25 is well-configured, used with sufficient retrieval depth, and paired with more capable LLMs in an agentic loop.

On the other hand, using a dense retriever does not necessarily translate into better answer quality. The system from [Meng et al. \(2026\)](#), which pairs gpt-5.2 with qwen3-embed-8b in row (e), achieves only 45.1% accuracy. In contrast, PI-SERINI with the same LLM, gpt-5.2, reaches 70.5% accuracy using BM25 in row (j). This gap suggests that final answer quality is not determined by retriever choice alone, but also by how the agent searches, manages context, and interacts with retrieved documents. The same pattern is also observed for the AgentIR-4B baseline in row (f).

Overall, these results address our research question in terms of final answer quality, showing that BM25 can achieve competitive answer accuracy in deep research when its parameters are well-configured, its retrieval depth is sufficient, and it is paired with more capable LLMs in an agentic loop.

We make two additional observations from the answer-quality results. First, the DeepSeek-based agents provide a strong cost-performance trade-off: deepseek-v4-flash achieves 68.1% accuracy with only \$28.9 total cost in row (n). Second, there is a substantial gap between claude-opus-4.7 and gpt-5.5, despite both being high-cost frontier models. claude-opus-4.7 achieves 69.8% accuracy in row (h), while gpt-5.5 reaches 83.1% in row (m). We further analyze their behavioral differences in Section 7.

Retrieval Behavior. Table 1 compares retrieval behavior across systems. For surfaced recall, all PI-SERINI variants substantially outperform the released baselines, with most runs achieving over 90% recall on both evidence and gold documents. Notably, gpt-5.5 + BM25 in row (m) reaches 94.7 and 94.4 on the evidence and gold documents, respectively, compared to 79.21 and 81.34 for the dense-retriever baselines of rows (d) and (f).

Under the same gpt-5 + BM25 setting, PI-SERINI also exposes more evidence to the agent. Compared with the released baseline in row (c), PI-SERINI in row (i) improves previewed recall of gold documents from 66.5 to 70.0, while maintaining similar previewed evidence-document recall. With gpt-5.5, (m) further achieves the highest previewed recall among BM25-based systems, reaching 73.6 for evidence documents and 72.9 for gold documents. Because agents can often synthesize answers from previewed excerpts without calling read_document, higher previewed recall need not yield proportional gains in behavior recall.

Overall, these results answer our research question from the perspective of retrieval effectiveness: a lexical retriever can retrieve the evidence needed for deep research when paired with more capable LLMs in an agentic loop, provided that BM25 is well-configured and used with sufficient retrieval depth. At the same time, the gap between surfaced and previewed recall shows that the remaining challenge lies in helping the agent navigate cached ranking and allocate context to the right evidence.

Cost Analysis. Prior work on BrowseComp-Plus has shown that deep research evaluation can be expensive (Chen et al., 2025; Meng et al., 2026). Meng et al. (2026), for example, report that one gpt-5.2 experiment in row (e) costs roughly \$1000–\$2000 USD because many queries require many iterations for an agent to reach its conclusion.

PI-SERINI substantially lowers this cost while preserving competitive performance. Under the same gpt-5 + BM25 setting, PI-SERINI reduces the cost from \$400.4 in the released baseline row (c) to \$94.9 in row (i). Even when paired with more expensive LLMs, PI-SERINI remains cost-efficient. For example, gpt-5.5 + BM25 in row (m) achieves the highest accuracy in the table at a cost of \$291.6, which is still lower than the released gpt-5 baselines in rows (c) and (d). Similarly, PI-SERINI with gpt-5.2 costs \$122.2, far below the roughly \$1000–\$2000 reported for the gpt-5.2 experiment in row (e).

These cost reductions matter because deep research studies depend on repeated evaluation. By lowering the cost of each full benchmark run, PI-SERINI makes ablations and future studies of deep research more practical. We further analyze the factors contributing to the cost savings in Section 6.

Tool Usage. Table 2 further compares how different search agents allocate their tool budget. The

System	Total	Search	Read	Browse
Chen et al. (2025)				
o3 + BM25	25.9	25.9	–	–
o3 + qwen3-embed-8b	24.0	24.0	–	–
gpt-5 + BM25	23.2	23.2	–	–
gpt-5 + qwen3-embed-8b	21.7	21.7	–	–
Chen et al. (2026)				
Tongyi-DR + AgentIR-4B	27.9	24.5	3.4	–
Meng et al. (2026)				
gpt-5.2 + qwen3-embed-8b	73.8	73.8	–	–
PI-SERINI (OURS)				
claude-haiku-4.5 + BM25	41.3	31.8	8.8	0.6
claude-opus-4.7 + BM25	9.0	6.8	2.1	0.1
gpt-5 + BM25	15.2	11.1	3.9	0.2
gpt-5.2 + BM25	17.2	12.6	3.9	0.7
gpt-5.4-mini + BM25	24.7	20.2	4.2	0.3
gpt-5.4 + BM25	23.1	17.9	4.3	0.9
gpt-5.5 + BM25	19.3	13.5	5.0	0.8
deepseek-v4-flash + BM25	33.2	26.1	7.0	0.1
deepseek-v4-pro + BM25	18.6	14.2	4.3	0.1

Table 2: Average tool calls per query for all systems

results show that, except for the claude-haiku-4.5-based agent, most PI-SERINI systems use a similar or smaller number of total tool calls compared to the BrowseComp-Plus released baselines. This pattern is especially clear under the same gpt-5 + BM25 setting. Compared with the released baseline, which uses 23.2 tool calls per query, PI-SERINI uses only 15.2. In contrast, the gpt-5.2 + qwen3-embed-8b system reported by Meng et al. (2026) uses a dense retriever but requires a substantially more tool calls, 73.8 per query, while not achieving better answer accuracy than PI-SERINI. This indicates that replacing dense retrievers with BM25 does not necessarily increase the number of tool calls, while still maintaining competitive retrieval effectiveness and answer accuracy, as shown in Table 1.

6 Ablation Study

Retrieval Depth. We vary the number of documents k returned by search to understand how much of the recall gain comes from increased retrieval depth.

Figure 2 shows that increasing k from 5 to 100 substantially improves surfaced recall from 70.5% to 86.22%. Previewed recall also improves over the same range, from 70.5% to 74.1%. When k is further increased to 1000, surfaced recall continues to rise, reaching 95.8%. In contrast, previewed recall does not continue to improve with larger k and reaches its highest value of 74.7% at $k = 50$.

These results show that retrieval depth is a major driver of surfaced recall in PI-SERINI. Increasing

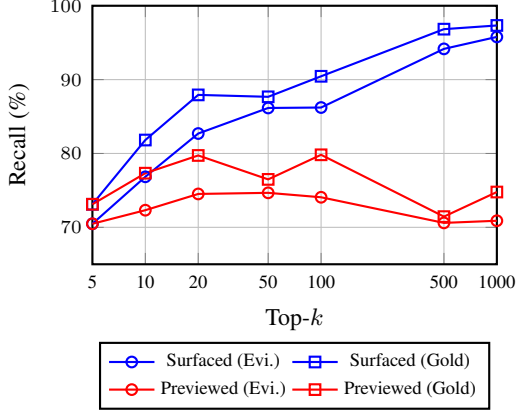


Figure 2: Effect of the number of documents (k) returned by the search tool in PI-SERINI with gpt-5.4

k gives BM25 enough first-stage capacity to place relevant evidence documents in the cached ranking, but the saturation of previewed recall suggests that larger cached rankings do not automatically lead the agent to inspect more relevant evidence.

Tuning BM25 for Long Documents. BM25 defaults are usually chosen for shorter documents, but BrowseComp-Plus documents are long and noisy. We therefore test whether PI-SERINI’s gains depend on retuning BM25 for this setting.

We sample 100 queries from BrowseComp-Plus and perform a grid search over k_1 and b . Figure 3 shows that ANSERINI’s default setting ($k_1 = 0.9, b = 0.4$) is placed in a low-performing region. Stronger configurations use larger k_1 and higher b , with the best setting near $k_1 = 16$ and $b = 1.0$. This pattern suggests that the default BM25 setting is underfit for long-document evidence search.

The first and second columns in Table 3 show the downstream effect on a 100-query BrowseComp-Plus subset. The default setting uses $k_1 = 0.9$ and $b = 0.4$, while the tuned setting uses $k_1 = 25$ and $b = 1$, following Table 1. We also tested the best grid-search setting, $k_1 = 16$ and $b = 1$, and observed similar results. Tuning raises accuracy from 64.0% to 82.0% (81.1% with the best grid-search setting) and surfaced recall from 84.6% to 95.7% (94.0% with the best grid-search setting). Previewed recall and behavior recall also improve, showing that better first-stage ranking gives the agent more useful evidence to inspect.

This sensitivity is unsurprising given the length of BrowseComp-Plus documents: the median document has $\sim 2k$ tokens, and the 90th percentile has $\sim 14k$ tokens (cf. Figure 4 in Chen et al. (2025)).

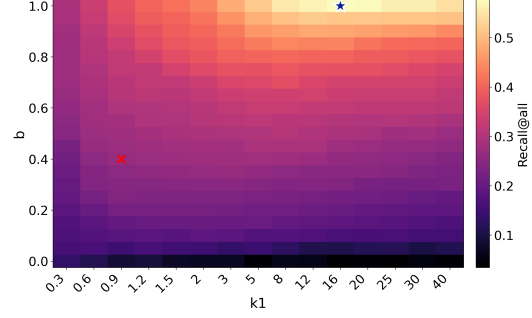


Figure 3: Grid search results of BM25 tuning over different k_1 and b combinations. $\star$ indicates the best-performing configuration, while $\times$ marks ANSERINI’s default setting: ($k_1 = 0.9, b = 0.4$).

At this length, relevant evidence is often buried in irrelevant text, making length normalization and term-frequency saturation matter more than in passage retrieval. ANSERINI’s default BM25 setting is tuned for that shorter-document regime, whereas tuned parameters better match long-document evidence search. This adaptation improves both evidence recall and final answer accuracy, consistent with the prior findings (Meng et al., 2026).

Cost-Efficiency To better understand which designs in PI-SERINI improve cost efficiency, we focus on three factors: (1) BM25 tuning, (2) termination policy, and (3) prefix cache, since these factors are the major differences between PI-SERINI and baseline systems.

We first examine the effect of BM25 tuning. The first and second columns of Table 3 show that, under the same termination policy, cost can be slightly reduced with a better retriever. This phenomenon is consistent with the baseline results in Table 1, from row (a) to row (d).

For the termination policy, we compare three settings: (1) a 300-second timeout, which is the standard setting in PI-SERINI; (2) a maximum of 100 iterations, which aligns with the baseline set-

Metric	Default BM25 (0.9, 0.4)	Tuned BM25	Tuned BM25 (25, 1)	Tuned BM25
BM25 (k_1, b)	Timeout 300s	Timeout 300s	MaxIter100	Timeout 3600s
Accuracy (%)	64.0	82.0	76.0	83.0
Acc. [†] (%)	66.7	83.7	92.7	83.0
Surfaced (%)	84.6	95.7	97.3	96.8
Previewed (%)	50.7	70.4	73.5	79.0
Behavior (%)	36.4	52.2	56.5	60.0
Avg. Tool Calls	22.6	23.1	27.3	27.1
Med. Total Tokens	281,324.0	271,140.5	263,880.5	218,085.0
Med. Cached Tokens	228,736.0	227,968.0	215,552.0	186,112.0
Cost (\$)	24.1	21.7	24.9	26.3

Table 3: Performance, cost, and tool-use comparison between default/tuned BM25 under different termination policies tested by PI-SERINI with gpt-5.4.

ting (Chen et al., 2025; Meng et al., 2026); and (3) a 3600-second timeout, which is an extremely large timeout used to simulate the absence of a practical termination policy. According to the second through last columns of Table 3, replacing the 100-iteration cap with the 300-second timeout slightly reduces cost, from \$24.9 to \$21.7, while substantially increasing the timeout to 3600 seconds does not dramatically increase the cost.

Regarding prefix cache, PI-SERINI inherits PI’s agentic loop, which structures the interaction history and enables prefix-cache-friendly configuration by default. Although Table 3 does not directly isolate the cost reduction from prefix caching, all settings exhibit high cached-token ratios: around 82% to 90% of total tokens are served from cache. Given the lower price of cached tokens, this suggests that the prefix-cache-friendly loop is a major contributor to PI-SERINI’s overall cost efficiency.

7 Discussion

Failure Mode Analysis. GPT-5.5 and Claude Opus 4.7 are frontier models with comparable inference costs in our setting, yet they exhibit a substantial performance gap on BrowseComp-Plus. Our trajectory analysis shows that both agents sometimes use internal knowledge or weakly related entities from retrieved documents to expand queries. A key behavioral difference is how the agent responds when such a probe is weak or incorrect. In cases where gpt-5.5 outperforms claude-opus-4.7, we observe that gpt-5.5 tends to keep candidate-specific probes reversible, returning to the original clues when they fail. Claude Opus 4.7, in contrast, often searches vertically within the same weak hypothesis. We refer to this failure mode as *premature branch commitment*. Intuitively, if a query asks for a cozy ramen shop near Tokyo station with a red lantern outside, both agents may probe a familiar but incorrect candidate such as *Ichiran*. If search agents cannot verify that *Ichiran* matches the original clues, gpt-5.5 would discard the hypothesis and return to the original clues, whereas claude-opus-4.7 would continue issuing *Ichiran*-related queries. The real case from BrowseComp-Plus is shown in Appendix C.

To Retrieve or Not To Retrieve. Recent work by Cao et al. (2026) and Li et al. (2026) shows that coding agents can achieve strong performance on BrowseComp-Plus by reformulating deep research as a file-system navigation problem. In their setup,

a coding agent interacts with a locally materialized subset of documents through command-line tools such as `rg`, `sed`, and `nl`, effectively reformulating a retrieval problem as a navigation problem via a file-based interface (cf. Figure 1 `codexw/cli`, `DCI-lite`, and `DCI-CC`; we refer readers to their papers for further details). However, this formulation assumes that the relevant search space can be localized in advance, which is difficult to satisfy in real-world settings where corpora are large, dynamic, and cannot be exhaustively stored or exposed. Notably, in the experimental setup of Cao et al. (2026), documents are randomly sampled to construct a roughly 100k-token working set in order to fit within the context window. Because of these differences in problem formulation, and because researchers sometimes run subsampled evaluations to reduce cost (Cao et al., 2026), we do not include a comparison in Table 1 and report their results only as reference points in Figure 1.

Nevertheless, their systems can be viewed as testing a setting where relevant documents are already available locally and the agent only needs to identify the needed context through simple interaction mechanisms. Our results show a complementary point that, with proper configuration and sufficient retrieval depth, BM25 can already provide a high-recall candidate set for the agent to inspect.

8 Conclusion

We asked: “Does a lexical retriever suffice as LLMs become more capable in an agentic loop?” Our results suggest that, on BrowseComp-Plus, a well-configured lexical retriever with sufficient depth can suffice for effective deep research when paired with more capable LLMs in an agentic loop. In our experiments, a well-configured lexical retriever with sufficient retrieval depth can match or exceed released dense-retriever baselines, while the prefix-cache-friendly agent harness makes full-benchmark runs far cheaper. These findings suggest that the low scores of previously released BM25 baselines mainly stem from ill-configured parameters and shallow retrieval depth, rather than from an inherent failure of lexical retrieval. The lesson is not that BM25 is enough everywhere, but that under-configured baselines can hide weak agent-retriever interaction and miss the opportunities in designing more effective and cost-efficient search agents.

For us as a community interested in continuing to push the boundaries of deep research and

information-seeking systems, the most useful signal is where PI-SERINI fails. It often surfaces the right evidence documents, but the agent, with its pretrained policies and behaviors in this era, does not always browse, open, or use them effectively. That failure mode shifts the question of system improvements from *Can the retriever find the evidence?* to *Can the agent recognize and spend context on the evidence it has already been given?* Future progress in deep research may therefore come less from pulling the retriever lever harder, and more from designing tools that help agents navigate evidence documents with better judgments.

9 Limitations

PI-SERINI shows that a well-configured BM25 retriever, combined with a carefully designed tool interface, can surface high-recall evidence. However, our study has several limitations. First, PI-SERINI still relies on a relatively minimal interface for navigating the cached ranking, resulting in substantially lower previewed and behavior recall than surfaced recall. Second, our evaluation is limited to BrowseComp-Plus, leaving open the question of whether the current agent harness generalizes to other settings, including multilingual queries and domain-specific scenarios. Finally, although using a timeout as the termination policy provides a simple way to control execution, queries with different levels of complexity may require different amounts of research time. A fixed time budget may therefore fail to balance effectiveness, latency, and monetary cost, leading either to insufficient exploration for difficult queries or unnecessary computation for simpler ones.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. [Self-RAG: Learning to retrieve, generate, and critique through self-reflection](#). In *Proceedings of the Twelfth International Conference on Learning Representations*.
- Weili Cao, Xunjian Yin, Bhuwan Dhingra, and Shuyan Zhou. 2026. [Coding agents are effective long-context processors](#). *Preprint arXiv:2603.20432*.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Jimmy Lin, Akari Asai, and Victor Zhong. 2026. [AgentIR: Reasoning-aware retrieval for deep research agents](#). *Preprint, arXiv:2603.04384*.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifmoghammad, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. 2025. [BrowseComp-Plus: A more fair and transparent evaluation benchmark of deep-research agent](#). *Preprint, arXiv:2508.06600*.
- Tiansheng Hu, Yilun Zhao, Canyu Zhang, Arman Cohan, and Chen Zhao. 2026. [SAGE: Benchmarking and improving retrieval for deep research agents](#). *Preprint arXiv:2602.05975*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). In *Proceedings of the 34th International Conference on Neural Information Processing Systems*.
- Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. [Search-o1: Agentic search-enhanced large reasoning models](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5420–5438.
- Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhui Chen, Jimmy Lin, Dongfu Jiang, and Yu Zhang. 2026. [Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction](#). *Preprint, arXiv:2605.05242*.
- Chuan Meng, Litu Ou, Sean MacAvaney, and Jeff Dalton. 2026. [Revisiting text ranking in deep research](#). In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru WANG, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2026. [ToolRL: Reward is all tool learning needs](#). In *Proceedings of 39th Annual Conference on Neural Information Processing Systems*.
- Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muenighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. 2025. [ReasonIR: Training retrievers for reasoning tasks](#). *Preprint, arXiv:2504.20595*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint arXiv:2402.03300*.
- Sahel Sharifmoghammad and Jimmy Lin. 2026. [Rerank before you reason: Analyzing reranking](#)

- tradeoffs through effective token cost in deep search agents. *Preprint, arXiv:2601.14224*.
- Zhengliang Shi, Yiqun Chen, Haitao Li, Weiwei Sun, Shiyu Ni, Yougang Lyu, Run-Ze Fan, Bowen Jin, Yixuan Weng, Minjun Zhu, Qiujie Xie, Xinyu Guo, Qu Yang, Jiayi Wu, Jujia Zhao, Xiaqiang Tang, Xinbei Ma, Cunxiang Wang, Jiaxin Mao, and 7 others. 2025. [Deep research: A systematic survey](#). *Preprint arXiv:2512.02038*.
- Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, Kuan Li, Liangcai Su, Litu Ou, Liwen Zhang, Pengjun Xie, Rui Ye, Wenbiao Yin, Xinmiao Yu, Xinyu Wang, and 38 others. 2025. [Tongyi DeepResearch technical report](#). *Preprint, arXiv:2510.24701*.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. [BrowseComp: A simple yet challenging benchmark for browsing agents](#). *Preprint, arXiv:2504.12516*.
- Xumeng Wen, Zihan Liu, Shun Zheng, Shengyu Ye, Zhirong Wu, Yang Wang, Zhijian Xu, Xiao Liang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. 2026. [Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs](#). In *Proceedings of the 14th International Conference on Learning Representations*.
- Ryen W. White. 2024. [Advancing the search frontier with AI agents](#). *Commun. ACM*, 67(9):54–65.
- Shirley Wu, Shiyu Zhao, Qian Huang, Kexin Huang, Michihiro Yasunaga, Kaidi Cao, Vassilis N. Ioannidis, Karthik Subbian, Jure Leskovec, and James Zou. 2024. [AvaTaR: Optimizing LLM agents for tool usage via contrastive reasoning](#). In *Proceedings of the Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Peilin Yang, Hui Fang, and Jimmy Lin. 2018. [Anserini: Reproducible ranking baselines using Lucene](#). *J. Data and Information Quality*, 10(4).
- Wei Yang, Kuang Lu, Peilin Yang, and Jimmy Lin. 2019. [Critically examining the "neural hype" weak baselines and the additivity of effectiveness gains from neural ranking models](#). In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval*, pages 1129–1132.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *Proceedings of the 11th International Conference on Learning Representations*.
- Mario Zechner and contributors. 2025. [PI-MONO: AI agent toolkit \(coding agent CLI, unified LLM API, TUI & web UI, Slack bot, vLLM pods\)](#). <https://github.com/badlogic/pi-mono>. GitHub repository.
- Wenlin Zhang, Kuicai Dong, Junyi Li, Yingyi Zhang, Xiaopeng Li, Pengyue Jia, Yi Wen, Derong Xu, Maolin Wang, Yichao Wang, Yong Liu, and Xiangyu Zhao. 2026. [To search or not to search: Aligning the decision boundary of deep search agents via causal intervention](#). In *Proceedings of the ACM Web Conference 2026*, page 2049–2059.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *Preprint, arXiv:2506.05176*.
- Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, Yuxin Gu, Sixin Hong, Jing Ren, Jian Chen, Chao Liu, and Yining Hua. 2025. [BrowseComp-ZH: Benchmarking web browsing ability of large language models in Chinese](#). *Preprint, arXiv:2504.19314*.

A System Setup

This appendix documents the exact PI-SERINI agent setup used in our experiments. We organize the setup by prompt, tool interface, backend configuration, and runtime policy. Placeholders in curly braces, such as `\{Question\}`, are replaced at runtime.²

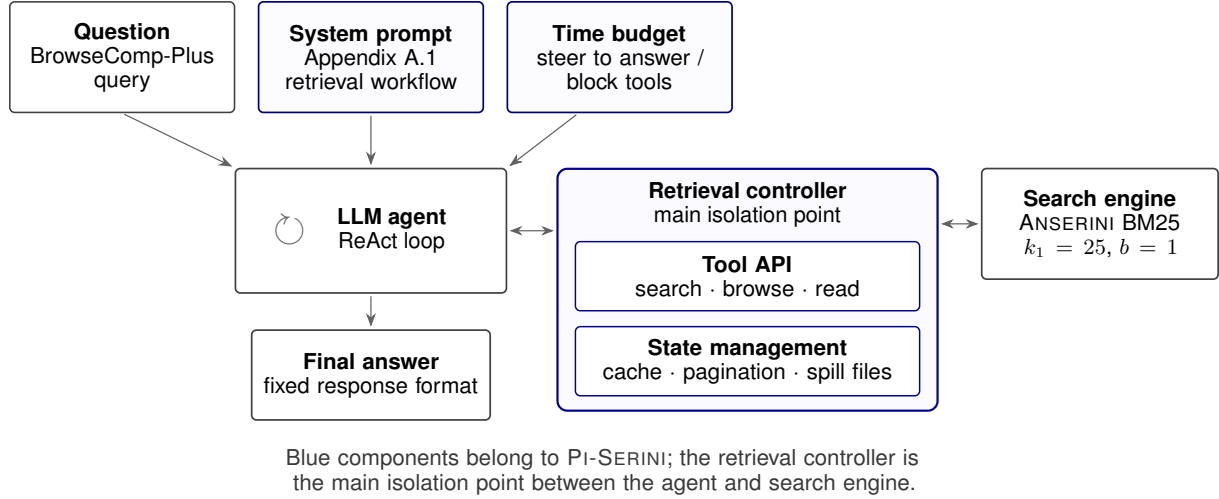


Figure 4: System architecture of PI-SERINI. The system includes the retrieval workflow prompt, time-budget steering policy, and retrieval controller. The LLM agent interacts with ANSERINI only through the controller, which exposes a constrained tool API and maintains local retrieval state for controlled, paginated BM25 retrieval.

A.1 Agent Prompt

Search Agent Prompt

You are a deep research agent answering a question using only the provided tools.

Workflow:

1. Use search with a concise raw query string based on the original question.
2. Prefer short lexical searches over long natural-language rewrites.
3. Browse the current ranking with `read_search_results` before repeatedly rewriting the query.
4. If a promising candidate document appears in the ranking, inspect it with `read_document`.
5. When reading a document, start with `offset=1` and a moderate limit. If it is truncated and still relevant, continue reading the same document.
6. Use search refinements only when they add a genuinely new clue from what you already saw.
7. Every call to `search`, `read_search_results`, and `read_document` must include `reason` as the first argument. Keep it specific, under 100 words, and focused on the clue, gap, candidate, or ranking issue.
8. As soon as you have enough evidence, stop using tools and answer in plain assistant text.
9. Your final response must use exactly this format:
 Explanation: {your explanation for your final answer. Cite supporting docids inline in square brackets [] at the end of sentences when possible, for example [123].}
 Exact Answer: {your succinct, final answer}
 Confidence: {your confidence score between 0% and 100%}
10. If you later receive a user steer telling you to submit now, stop using tools immediately and answer right away with the exact final response format below. Do not do more research after that steer.
11. Keep Exact Answer concise and directly responsive to the question.

Question: {Question}

²The setup documented in this appendix corresponds to PI-SERINI’s codebase at commit 68c5e0f.

A.2 Retrieval Tool Interface

Tool: search

Purpose: Send a raw lexical query to the configured retrieval backend.

Arguments:

reason: brief rationale, supplied first, at most 100 words
query: raw query string; not a structured object and not raw Lucene syntax

Runtime behavior in our BM25 runs:

query_mode = plain
backend request limit = 1000 hits
initial displayed page = ranks 1-5

Output:

search_id, formatted first page, cached docids, displayed docids, timing/truncation metadata

Tool: read_search_results

Purpose: Browse an existing cached ranking by search_id without issuing a new backend query.

Arguments:

reason: brief rationale, supplied first, at most 100 words
search_id: identifier returned by search
offset: optional 1-indexed rank offset; default is 6
limit: optional number of ranked hits to show; default is 10

Output:

formatted page from the cached ranking, displayed docids, next_offset when more hits remain,
and truncation metadata if the formatted page is too large

Tool: read_document

Purpose: Read one backend document by docid in line-based chunks.

Arguments:

reason: brief rationale, supplied first, at most 100 words
docid: document identifier from search or read_search_results
offset: optional 1-indexed line offset; default is 1
limit: optional maximum number of lines; default is 200

Output for a found document:

formatted document lines, returned line range, total line count, backend truncation flag,
next_offset when available, timing metadata, and output-truncation metadata when applicable

A.3 Backend and Cache Configuration

Retrieval Backend Configuration

Experimental backend used in this paper:

retriever = \textsc{Anserini} BM25
corpus = BrowseComp-Plus
BM25 parameters = k1 25, b 1

pi-search configuration contract:

PI_SEARCH_EXTENSION_CONFIG is required and must be supplied by the caller
our experiments use \textsc{Anserini}'s BM25 backend kind

Backend kinds supported by the pi-search config schema:

- \textsc{Anserini}-bm25 over TCP: {backend: {kind: "\textsc{Anserini}-bm25", transport: {kind: "tcp", host, port}}}
- \textsc{Anserini}-bm25 over stdio: {backend: {kind: "\textsc{Anserini}-bm25", transport: {kind: "stdio", indexPath}}}
- http-json: {backend: {kind: "http-json", capabilities, endpoints}}
- mock: {backend: {kind: "mock", documents}}

Search Cache, Pagination, and Spill Files

Search cache:

each search call receives a session-local search_id such as s1, s2, ...
each search_id stores the raw query, query mode, and retrieved ranking
at most 32 search_id entries are retained; older entries are evicted first

Ranking depth and pagination:

search requests up to 1000 backend hits and caches the returned ranking
search displays only ranks 1-5 immediately
read_search_results defaults to offset=6 and limit=10 when omitted
read_document defaults to offset=1 and limit=200 lines when omitted

Spill-file behavior:

tool output is first formatted for the agent context
if formatted output exceeds Pi's line or byte truncation limits, the visible
output is truncated and the complete search page or document chunk is saved
under a temporary pi-search spill directory
the temporary spill directory is cleaned up on session shutdown or process exit

A.4 Time-Budget Policy

Submit-Now Steering Policy

```
Per-query timeout in the main experiments: T = 300 seconds
Activation condition:
  the timer is enabled only when TIMEOUT_SECONDS is set to a positive finite value
Submit-now trigger:
  delay = floor(0.7 * TIMEOUT_SECONDS * 1000) milliseconds
Submit-now behavior when the timer fires while the agent is active:
  1. Send a user steer saying the time budget is nearly exhausted.
  2. Instruct the agent to stop using tools and submit its best answer immediately.
  3. Block later calls to search, read_search_results, and read_document.
Final response format after the steer remains:
  Explanation: ...
  Exact Answer: ...
  Confidence: ...
```

A.5 Gold-Answer LLM Judge

We evaluate final-answer correctness with a gold-answer LLM judge. The judge receives the question, the agent’s final response, and the benchmark-provided correct answer. The default judge model is `openai-codex/gpt-5.3-codex`, run through PI in JSON mode. Each judge call uses an isolated `PI_CODING_AGENT_DIR`, writes raw judge events and stderr logs, and times out after 180 seconds.

Gold-Answer Judge Configuration

```
Pi invocation:
  pi --no-tools --no-session --no-skills --mode json \
    --model openai-codex/gpt-5.3-codex --thinking low {judge_prompt}
Judge timeout: 180 seconds per query
Evaluation output root: evals/pi_judge
Ground-truth file:
  data/browsecomp-plus/ground-truth/browsecomp_plus_decrypted.jsonl
Per-query outputs:
  per-query/{query_id}_eval.json
  raw-events/{query_id}.jsonl
  stderr/{query_id}.log
```

Gold-Answer Judge Prompt

```
You are an evaluation judge.

Your job is to determine whether the response's final answer is semantically equivalent to the known correct answer.
Do not solve the question yourself.
Do not use outside knowledge.
Focus only on whether the response's final answer matches the correct answer.
Allow harmless wording differences, equivalent formatting, and added correct detail.
For numerical answers, allow small formatting differences and obvious equivalent forms.
If the response does not contain a final answer you can extract, set extracted_final_answer to null and correct to false.

Return exactly one JSON object and nothing else.
Do not wrap the JSON in markdown or code fences.
Use this exact schema:
{
  "extracted_final_answer": string | null,
  "correct_answer": string,
  "reasoning": string,
  "correct": boolean,
  "confidence": number
}

Requirements:
- confidence must be a number between 0 and 100
- correct must be true or false
- repeat the provided correct answer exactly in correct_answer
- reasoning must explain only whether the extracted final answer matches the correct answer

Question: {question}

Response:
{response}

Correct answer: {correct_answer}
```

B Model Token Pricing Used in Benchmark Runs

Table 4 reports the token pricing used by our benchmark runtime. All prices are in **USD per 1M tokens**.

Table 4: Token pricing used during benchmark execution. All prices are in USD per 1M tokens.

Model	Input	Output	Cache Read
claude-haiku-4.5	1.00	5.00	0.10
claude-opus-4.7	5.00	25.00	0.50
gpt-5	1.25	10.00	0.125
gpt-5.2	1.75	14.00	0.175
gpt-5.3-codex	1.75	14.00	0.175
gpt-5.4-mini	0.75	4.50	0.075
gpt-5.4	2.50	15.00	0.25
gpt-5.5	5.00	30.00	0.50
deepseek-v4-flash	0.14	0.28	0.028
deepseek-v4-pro	1.74	3.48	0.145

C Premature Branch Commitment

This example shows the query trajectories for Query 678 and demonstrates that incorrect query expansion is not inherently fatal. GPT-5.5 also introduces unverified candidate terms, such as **Warrington**, **Vinegar Strokes**, and **Arts Educational**, which lead to incorrect search directions. However, these probes remain reversible: after unproductive searches, GPT-5.5 returns to the root constraints, such as the town population and spelling-history clue, and then follows the evidence-supported “Crawley” path. This eventually leads to the correct candidate, “Dani Sylvia,” and the query is refined using the original interview constraints.

Claude Opus 4.7 follows a different pattern. After the first search retrieves a title mentioning **Renato Paris** and **CICAS**, the agent treats this superficial match as a candidate anchor. It then searches vertically within the Renato Paris branch, adding more unverified terms such as **Tom Misch**, **Italy**, **Forlì**, **ICMP**, **BIMM**, and **Berklee**. These queries deepen the same hypothesis rather than testing alternative interpretations of the original clues. The issue is therefore not that Claude uses domain knowledge, but that it keeps elaborating a weak branch after the supporting evidence is insufficient.

Original Query

Find the school mentioned in a November 2019 interview by an artist who grew up in a town with 2019 population around 114,000 and current spelling appearing around 1316; performed two roles in the same industry; won a late-2010s award; released a creation in 2019; described their style as “eclectic”; described social media as “dangerous”; and said Instagram or Twitter were the best way to connect.

GPT-5.5 Query Trajectory

Root: Original query

```
-- Step 1
| Reason: Locate the November 2019 interview containing the exact quoted style/social media phrases.
| Query: "eclectic" "social media is dangerous" "best way to connect" interview November 2019 artist
|
-- Step 2
| Reason: Search exact dangerous quote with November interview lexical clues, dropping broader social phrase.
| Query: "social media is dangerous" "Instagram" "Twitter" "interview" "2019" artist
|
-- Step 3
| Reason: Find pages with both eclectic self-description and contact via social platforms from music interviews.
| Query: "How would you describe your style" "eclectic" "social media" "dangerous"
|
-- Step 4
| Reason: Search exact interview question about style with release-year clue likely in independent artist Q&A.
| Query: "describe your style as eclectic" "interview" "2019" musician
|
-- Step 5
| Reason: Target the requested answer phrase in the interview: artist says social media dangerous and mentions training school.
| Query: "social media" "dangerous" "trained at" "eclectic" "interview"
|
-- Step 6
| Reason: Exact dangerous phrase alone may reveal the interview despite noisy broad results.
| Query: "social media is dangerous"
|
-- Step 7
| Reason: Look for variant wording around social media danger plus eclectic in artist interviews.
| Query: "social media" "dangerous" "eclectic" "Instagram" "interview" "November"
|
-- Step 8
| Reason: Use the town clue Warrington plus exact social-media/interview terms to identify the artist.
| Query: "grew up in Warrington" artist "eclectic" "social media" "dangerous"
|
-- Step 9
| Reason: Search Warrington artist and 2019 interview clue without exact dangerous phrase.
```

```

|       Query: "grew up in Warrington" "interview" "eclectic" "2019"
|-- Step 10
| Reason: Identify town from population 114,000 and name-spelling clue.
| Query: "current spelling" "first appeared" "1316" town population 114000
|
|   |-- Step 11
|   | Reason: Use identified town Crawley with the interview quote to find the artist.
|   | Query: "grew up in Crawley" "social media" "dangerous" "eclectic"
|   |
|   |-- Step 12
|   | Reason: Search Crawley plus November 2019 interview and eclectic without exact grew-up phrase.
|   | Query: "Crawley" "eclectic" "social media" "interview" "November" "2019" artist
|   |
|   |-- Step 13
|   | Reason: Candidate from Crawley clue: Vinegar Strokes; verify interview wording and training school.
|   | Query: "Vinegar Strokes" "eclectic" "social media" "dangerous" "trained"
|   |
|   |-- Step 14
|   | Reason: Find candidate biography documents for Vinegar Strokes and relevant facts.
|   | Query: "Vinegar Strokes" "grew up in Crawley" "Arts Educational"
|   |
|-- Step 15
| Reason: Candidate Dani Sylvia matches Crawley, 2019 release, songwriter award; locate November 2019 interview with school answer.
| Query: "Dani Sylvia" "eclectic" "social media" "dangerous" "school"

```

Claude Opus 4.7 Query Trajectory

```

Root: Original query

|-- Step 1
| Reason: Find the November 2019 interview where artist calls style "eclectic" and social media "dangerous".
| Query: interview November 2019 style "eclectic" social media "dangerous"
|
|   |-- Step 2
|   | Reason: Find the actual Renato Paris CICAS interview content.
|   | Query: Renato Paris CICAS interview eclectic dangerous
|   |
|   |   |-- Step 4
|   |   | Reason: Find info about Renato Paris artist hometown school training.
|   |   | Query: Renato Paris musician singer producer hometown
|   |   |
|   |   |-- Step 5
|   |   | Reason: CICAS podcast about Renato Paris.
|   |   | Query: CICAS podcast Renato Paris keyboardist
|   |   |
|   |   |-- Step 6
|   |   | Reason: Find Renato Paris biography or interview content elsewhere in corpus.
|   |   | Query: Renato Paris interview eclectic style
|   |   |
|   |   |-- Step 7
|   |   | Reason: Search for Renato Paris bio---he might be a different artist.
|   |   | Query: "Renato Paris" biography born
|   |   |
|   |   |-- Step 10
|   |   | Reason: Renato Paris Tom Misch keyboardist Forli.
|   |   | Query: Renato Paris Tom Misch keyboards Italy
|   |   |
|   |   |-- Step 11
|   |   | Reason: Forli Italy population history spelling.
|   |   | Query: Forli Wikipedia population history name origin
|   |   |
|   |   |   |-- Step 12
|   |   |   | Reason: Look for Forli Wikipedia article in the corpus.
|   |   |   | Query: Forli city Italy Emilia-Romagna
|   |   |   |
|   |   |   |-- Step 16
|   |   |   | Reason: Try different angle---artist from Forli grew up there.
|   |   |   | Query: "grew up in Forli" musician artist
|   |   |   |
|   |   |   |-- Step 17
|   |   |   | Reason: Try searching for transcription of the interview content with Italian connection.
|   |   |   | Query: Renato Paris singer Italian Forli conservatory
|   |   |   |
|   |   |-- Step 13
|   |   | Reason: Search general corpus for Renato Paris music school training.
|   |   | Query: Renato Paris trained school music London
|   |   |
|   |   |-- Step 15
|   |   | Reason: CICAS abbreviation context---what is it.
|   |   | Query: CICAS podcast meaning organization music creative
|   |   |
|   |   |-- Step 18
|   |   | Reason: Look for Renato Paris ICMP or BIMM or Tech Music School.
|   |   | Query: "Renato Paris" ICMP BIMM Tech Music School Berklee Trinity
|   |   |
|-- Step 8
| Reason: Try finding the exact interview piece via different phrasing.
| Query: "my style is eclectic" interview "dangerous" Instagram Facebook
|
|-- Step 9
| Reason: Search for specific quoted phrases from a November 2019 artist interview.
| Query: "social media is dangerous" interview 2019 musician eclectic
|
|-- Step 14
| Reason: Try finding interview with specific phrases about connecting via social media.
| Query: "best way to connect" Instagram Twitter interview eclectic 2019

```